Use Case Description 1

Identifier	UC-01
Name	Upload Assessment
Summary	This use case allows the evaluator to upload assessments onto the website for his/her students.
Priority	High
Actors	Evaluator
Pre-condition(s)	The evaluator must be logged into the system. There should be a class that exists for him to upload the assignment.
Post-condition(s)	Assessment is successfully uploaded on the website and is viewable by the students of the class.
Dependencies	Depends on the evaluator having a class.
Exceptions	The system fails to upload the assessment due to server issues.

Typical Course of Action

S #	Actor Action	System Response
1	Evaluator opens the “Upload Assessment” option.	The system opens the assessment uploading interface.
2	Evaluator creates a new assessment by adding details such as title and description.	The system verifies the details and updates the assessment section for respective students.
3	Evaluator edits the details on the uploaded assessment.	The system updates it respectively.
4	Evaluator deletes the assessment	The system removes the task and updates the list respectively.

5	Evaluator organises the assessment according to marked and unmarked.	The system arranges data accordingly.
Alternate Course of Action		
S #	**Actor Action**	**System Response**
2a	Evaluator enters incomplete or invalid assessment details.	The system prompts the host to correct the details.
3a	Evaluator attempts to edit a non-existent or deleted assessment.	The system displays an error message.
4a	The evaluator tries to delete an assessment where the deadline has already passed.	The system displays a warning and prevents deletion.

Use Case Description 2

Identifier	UC-02
Name	Add Rubric
Summary	This use case describes that the evaluator can add a specific rubric that the students need to follow.
Priority	Medium
Actors	Evaluator
Pre-condition(s)	Assessment should already be uploaded.
Post-condition(s)	The rubric should be visible to all students of the class.
Dependencies	Depends on Upload Assessment as assessment must exist first.
Exceptions	The system fails to add the rubric due to server issues.

Typical Course of Action		
S #	**Actor Action**	**System Response**
1	The evaluator adds the rubric in sections for the assessment for guidance.	The system updates the respective assessment with the rubric.

Alternate Course of Action		
S #	**Actor Action**	**System Response**

1a	The evaluator tries adding rubric for a non-existing assessment.	The system shows an error.

Use Case Description 3

Identifier	UC-03
Name	Add Deadline
Summary	This use case allows the evaluator to add a deadline to the assessments uploaded to ensure that the students submit the work on time.
Priority	Medium
Actors	Evaluator
Pre-condition(s)	Assessment should already be uploaded. The deadline should be on or after the upload date.
Post-condition(s)	Assessments are either not accepted or marked as late if not submitted within the deadline.
Dependencies	Depends on Upload Assessment ensuring that there is an assessment that can be restricted with a deadline.
Exceptions	The system fails to add the deadline due to server issues.

Typical Course of Action		
S #	**Actor Action**	**System Response**

1	The evaluator adds the deadline for the assessment to ensure timely submission.	The system updates the respective assessment with the deadline.
		Adds assessment deadline to calendar.
Alternate Course of Action		
S #	**Actor Action**	**System Response**
1a	The evaluator tries adding a deadline for a non-existing assessment.	The system shows an error.
1b	The evaluator tries adding a deadline with a date that has already passed.	The system displays a warning.

Use Case Description 4

Identifier	UC-04
Name	Upload Key
Summary	The evaluator can upload the key to the assessment uploaded which can only be viewed by the students after the deadline.
Priority	Low
Actors	Evaluator
Pre-condition(s)	Key can only be uploaded for an assessment that has already been uploaded on the website.
Post-condition(s)	The key should not be visible till the deadline.
Dependencies	Depends on Upload Assessment since the key cannot be added to an assessment that has not yet been uploaded.
Exceptions	The system fails to upload the key due to server issues.
Typical Course of Action	

S #	Actor Action	System Response
1	The evaluator uploads the key for the assessment to help students self-evaluate.	The system shows the key after the deadline.
Alternate Course of Action		
S #	**Actor Action**	**System Response**
1a	The evaluator tries adding a key for a non-existing assessment.	The system shows an error.
1b	The evaluator tries publishing the key publicly before the deadline.	The system displays a warning.

Use Case Description 5

Identifier	UC-05
Name	Evaluate Documents
Summary	The evaluator can directly mark the assessments on the website itself, allowing easier handling of student submissions.
Priority	High
Actors	Evaluator
Pre-condition(s)	Student submissions must exist.
Post-condition(s)	Marks given are visible to the respective students.
Dependencies	Depends on student submissions, as the evaluator can only mark submitted work.
Exceptions	The system fails to update the document marks after evaluation due to server issues.

Typical Course of Action		
S #	**Actor Action**	**System Response**
1	The evaluator uses the rubric to mark the student.	The system reflects the marks awarded for the respective student.
2	The evaluator may negatively mark students for cases of plagiarism or cheating.	The system shows the negative marks separately to the respective student.
Alternate Course of Action		
S #	**Actor Action**	**System Response**
1a	The evaluator gives marks less than zero to a student.	The system gives a warning which the evaluator can override.

Use Case Description 6

Identifier	UC-06
Name	Display Feedback
Summary	Any feedback by the evaluator about the assessment is displayed privately to the respective student.
Priority	Low
Actors	Evaluator, System
Pre-condition(s)	The student has already submitted the assessment.
Post-condition(s)	The feedback is only privately displayed to the student.
Dependencies	Depends on whether the student has made a submission. Depends on the System actor to display the feedback to the student.
Exceptions	The system fails to display and update the feedback for

	the student due to server issues.	
Typical Course of Action		
S #	**Actor Action**	**System Response**
1	The evaluator comments on a student's work.	The system reflects this comment by displaying it to the respective student.
Alternate Course of Action		
S #	**Actor Action**	**System Response**
1a	The evaluator tries to direct comment to a non-existing student.	The system displays an error.

Use Case Description 7

Identifier	UC -07
Name	Upload Code File
Description	Cpp code file(s) can be dragged and dropped or simply uploaded for an assigned task demanding cpp file(s)
Priority	High
Actors	Student
Pre-condition(s)	1. The student should have an account 2. The student should be logged in 3. The student should be connected with an evaluator 4. If there exists a deadline for the assigned task, it should not have been passed
Post-condition	1. The uploaded file(s) should be of cpp format

(s)	
Dependency	Dependent on existence of an assignment and its deadline, not being passed
Exceptions	Internet signal lost while trying to upload the code

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Student clicks on upload the code corresponding to the assigned task	System gives option to the user to either drag and drop a file or upload from PC
2.	Student clicks drag and drop	System provides an interface required for drag and drop
3.	Student drops the file	Students have the choice to submit another cpp file or click submit.
4.	Student clicks submit	The status of the assigned task appears to be submitted
		The submitted file appears on the respective evaluator's dashboard
5.	A student may unsubmit a code file	The status again shows to be unsubmitted until student resubmits a code file
		The file appearing on evaluators' dashboard should disappear from there until student resubmits a code file

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Student clicks upload from PC	The file explorer of that PC opens
3a	Student selects the cpp file	The selected file gets uploaded and the same course of action is followed as stated earlier
3b	Uploaded file not a pdf file	Prompts to upload the correct format
1a	Deadline has passed	The system displays message not

		allowing to upload any file

Use Case Description 8

Identifier	UC -08
Name	Upload Document
Description	Document can be dragged and dropped or simply uploaded for an assigned task demanding a pdf file.
Priority	High

Actors	Student
Pre-condition(s)	1. The student should have an account 2. The student should be logged in 3. The student should be connected with an evaluator 4. If there exists a deadline for the assigned task, it should not have been passed
Post-condition (s)	1. The uploaded file(s) should be of pdf format
Dependency	Dependent on existence of an assignment and its deadline, not being passed
Exceptions	Internet signal lost while trying to upload a document

Typical Course of Action

S#	Actor Action	System Response
1.	Student clicks on upload document corresponding to the assigned task	System gives option to the user to either drag and drop a file or upload from PC
2.	Student clicks drag and drop	System provides an interface required for drag and drop
3.	Student drops the file	Students have the choice to submit another cpp file or click submit.
4.	Student clicks submit	The status of the assigned task appears to be submitted
		The submitted file appears on the respective evaluator's dashboard
5.	A student may unsubmit a code file	The status again shows to be unsubmitted until student resubmits a code file
		The file appearing on evaluators' dashboard should disappear from there until student resubmits a code file

Alternate Course of Action

S#	Actor Action	System Response
2a	Student clicks upload from PC	The file explorer of that PC opens

3a	Student selects the pdf file	The selected file gets uploaded and the same course of action is followed as stated earlier
3b	Uploaded file not a pdf file	Prompts to upload the correct format
1a	Deadline has passed	The system displays message not allowing to upload any file

Use Case Description 9

Identifier	UC -09
Name	Enrol in course
Description	To connect to an evaluator (get enrolled in a course) the student needs to enter a unique code assigned by the

	evaluator for that course
Priority	Medium
Actors	Student
Pre-condition(s)	1. The student should have an account 2. The student should be logged in
Post-condition (s)	1. The student gets included in a course (a new course appears inside the current semester)
Dependency	Dependent on existence of a student account
Exceptions	Internet signal lost while trying to enrol in a course

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Student clicks on his current semester	System shows the list of courses it is included in already
2.	Student clicks add course	System prompts to type in a code
3.	Student enters the code	A new course appears in the current semester with its evaluator(s)
		A new student is added to the list on the evaluator(s) dashboard

Alternate Course of Action		
S#	**Actor Action**	**System Response**
3a	Student enters incorrect code	Error message is displayed

Use Case Description 10

Identifier	UC -10
Name	Self Evaluate code
Description	Student can self evaluate his code using the provided rubric, test cases and key

Priority	Low
Actors	Student
Pre-condition(s)	1. The student should have an account 2. The student should be logged in 3. The deadline should have been passed for the assignment 4. The code file should be of cpp format 5. The rubric, key and test cases should be visible to the student
Post-condition (s)	
Dependency	Dependent on termination of assessment deadline

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Student clicks on self evaluation once the deadline passes	System displays the test cases for which the student can run his code
		System also displays a copy of the rubric uploaded by evaluator previously.
2.	Student can run his code on the given test cases and give partial markings on the editable version of the rubric. And can also compare the solution code with key uploaded by the evaluator after deadline passes.	Partial marks are displayed on the self-evaluation tab of the assignment

Use Case Description 11

Identifier	UC -11
Name	Self Evaluate document

Description	The student can evaluate his solution pdf by comparing it with the solution key and rubric
Priority	Low
Actors	Evaluator
Pre-condition(s)	1. The student should have an account 2. The student should be logged in 3. The deadline should have been passed for the assignment 4. The doc file should be of pdf format 5. The rubric and key should be visible to the student
Post-condition (s)	
Dependency	Dependent on termination of assessment deadline

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Student clicks on self evaluation once the deadline passes	System displays the uploaded solution pdf
		It also displays the key and editable rubric
2	Students give partial marking on the editable version of the rubric. And can also compare the solution code with key uploaded by the evaluator after deadline passes.	Partial marks are displayed on the self-evaluation tab of the assignment

Use Case Description 12

Identifier	UC -12
Name	Sign up
Description	Sign up as an evaluator or a student with a unique gmail id
Priority	High
Actors	Evaluator, Student
Pre-condition(s)	1. The gmail id being used to make an account should not exist in the website's database (should be unique) 2. A password will be kept by the user for future log in
Post-condition (s)	1. A password will be assigned for ensuring security to the dashboard 2. A dashboard unique to the user, should be assigned to him
Dependency	Dependent on existence of a gmail address not added previously
Exceptions	Internet signal lost while trying to make an account

Typical Course of Action

S#	Actor Action	System Response
1	Make an account on the website as an "evaluator" or "student"	System asks for a unique gmail id and to set a password
2	Users fill in the required credentials	System adds the user to the database and provides him with a dashboard/account

Alternate Course of Action

S#	Actor Action	System Response
2a	Entered gmail address already exists	Prompts to make account using some other gmail address or login through the existing gmail

Use Case Description 13

Identifier	UC -13
Name	Generate course
Description	Evaluator generates a course and assigns its unique codes to students and other evaluators
Priority	High
Actors	Evaluator
Pre-condition(s)	1. The evaluator account should exist 2. The evaluator should be logged in
Post-condition (s)	1. A code for adding students will be provided 2. A code for adding another evaluator(TA) will be provided
Dependency	Dependent on existence of an evaluator account
Exceptions	Internet signal lost while trying to generate a course

Typical Course of Action

S#	Actor Action	System Response
1	Evaluator clicks generate course	System displays a new course and provides codes that will remain stored alongside the course
2	Student enters students' code	Student gets enrolled into the course
3	Another evaluator enters evaluator's code	He becomes an evaluator for this course alongside the one who generated it.

Use Case Description 14

Identifier	UC-14
Name	Login Verification
Summary	This use case describes the procedure for verifying the account via email id and password of the user stored in our database, in order to grant access to the site.
Priority	High
Actors	System, User (Evaluator, Student)
Pre-condition(s)	The user account should exist
Post-condition(s)	The user is granted access as an evaluator or student, they can enter respective classes/sections afterwards
Dependencies	User accounts must already be created so they can login later
Exceptions	Login fails due to connection issues.

Typical Course of Action		
S #	**Actor Action**	**System Response**
1	User selects "Log in to account" option available on website	Login screen pops up
2	User is prompted to enter credentials (email id, password)	Compares entered information with database data
3	User logs in successfully	Redirected to respective home page of the user

Alternate Course of Action		
S #	**Actor Action**	**System Response**

1a	User enters invalid credentials	Error message “Invalid email ID/password” pops up
		User is prompted to enter details again

Use Case Description 15

Identifier	UC-15
Name	Evaluate Code
Summary	This use case describes the procedure for compiling the code file, showing errors, comparing it with pre-defined test cases and assigning partial marks per test case.
Priority	High
Actors	System
Pre-condition(s)	1- The coding assessment must be uploaded 2- Evaluator must define test cases 3- Rubric section with marks for ‘correctness’ defined 4- Solution code file uploaded by student
Post-condition(s)	Code is evaluated on the basis of test-cases, total marks appear in the ‘correctness’ section of rubric, evaluators add ‘logic’, ‘efficiency’ marks
Dependencies	Code submission (by uploading file) must be done so it can be evaluated by the system. Test cases must also be pre-defined by the evaluator
Exceptions	Code autograding disrupted due to system error

Typical Course of Action		
S #	**Actor Action**	**System Response**
1	Evaluator (Teacher/TA) uploads assessment for coding	New assessment shows on student’s home page

2	Evaluator sets test cases for every coding question	Only shown on teacher's screen; hidden to students (until they select self-evaluation option after deadline)
3	Rubric section added for various aspects including 'correctness' defined	Visible on student's screen in rubric section
4	Evaluator selects 'autograde' option	Compiler used to compile code
		Test on pre-defined test cases
		Assign and display partial marks per test case
		Display cumulative marks in 'correctness' section of rubric
5	While checking, the evaluator adds logic, efficiency or other relevant marks in respective rubric sections	
Alternate Course of Action		
S #	**Actor Action**	**System Response**
4a	Code has errors	Display them

Use Case Description 16

Identifier	UC-16
Name	Display Charts
Summary	This use case describes the procedure for viewing analytics based on the submissions and evaluations in the form of tables or simple charts.
Priority	Medium
Actors	System
Pre-condition(s)	History of previously evaluated/submitted work must be maintained
Post-condition(s)	Students use charts for self-reflection and improvement, evaluators use them for tracking progress of students
Dependencies	History of data must be maintained to allow relevant calculations and visualization
Exceptions	Internet connectivity lost; chart designing disrupted

Typical Course of Action

S #	**Actor Action**	**System Response**
1	User selects 'visualize progress' option	Prompts user to enter chart type: bar chart, pie chart

2	Selects type of chart	Displays options for chart: average marks of all students per assessment, student performance trend across assessments, submission status trend
3	Selects option for data to be displayed on the chart	Designs chart; uses history maintained in the database for relevant calculations
		Displays chart to user
4	Uses chart to analyze relevant areas and make improvements/decisions accordingly	
Alternate Course of Action		
S #	**Actor Action**	**System Response**
3a	User selects an option for which inadequate data is available	Error message 'Insufficient data' displayed
		User prompted to select another option, else exit

Use Case Description 17

Identifier	UC-17
Name	Detect Plagiarism
Summary	This use case describes the procedure for detecting plagiarism in submissions using our database, by comparing them with old documents submitted.
Priority	High
Actors	System
Pre-condition(s)	History of previously submitted work must be maintained
Post-condition(s)	Submission flagged as plagiarized, plagiarism percentage shown, visible along with submission to evaluator
Dependencies	Submission history must be maintained to allow comparison with previously submitted files
Exceptions	Notification service unavailable/ flagging fails

Typical Course of Action		
S #	**Actor Action**	**System Response**
1	Student submits solution file (pdf)	Visible to evaluator
2	Evaluator selects 'detect plagiarism' option	Prompts user to enter minimum percentage required to flag
3	Evaluator sets flagging percentage for plagiarism, selects 'start'	Runs plagiarism check
		Submitted file is compared with previous submissions
		Similarity percentage calculated
		If calculated percentage >= evaluator's defined percentage, submission flagged as plagiarized
		Plagiarism percentage displayed to evaluator along with tag
Alternate Course of Action		
S #	**Actor Action**	**System Response**
2a	Evaluator selects 'detect plagiarism' option for a course with no previous submission history	Error message "Insufficient submission history to detect plagiarism" displayed
		Prompts user to exit